MEETING FIELD GUIDE / 01

Your RocketRide meeting brief

Redis node integration and app proposal | October 29, 2026 | Prepared for Aiden Buck

The goal today: bring an informed first proposal, show what you explored, and leave with explicit decisions about the node, app, owners, and schedule. This is a discussion draft, not a record of RocketRide approval.

Understand the project in one minute

RocketRide lets developers assemble AI workflows from reusable components called nodes. Your team will add a Redis node so workflows and agents can read and write fast, shared data. You will also build an app that demonstrates why that node is useful.

RocketRide supplied the project PDF first. Its later Discord message asks your team to review that brief, explore the product and code, draft a scope, bring two or three app ideas, and assign task owners. Use their brief as the starting point for discussion.

The engagement has three deliverables

Deliverable	What it means
Redis node	Code, tests, configuration, and README contributed to RocketRide's catalog through a reviewed pull request.
App	A user-facing RocketRide App Builder app whose core feature uses the new node. Answer Cache is their proposed idea.
Demo and outreach	A live demo, technical blog, community posts, and later social clips. RocketRide handles specified publishing and video work.

Read this packet in this order

Pages 2-3: understand RocketRide and the node proposal. **Pages 4-5:** choose an app and understand the cache. **Pages 6-7:** testing, ownership, and milestones. **Page 8:** use during the call. **Pages 9-10:** inspect the code and check sources.

The client brief proposes design approval on October 30, a PR on November 6, merge on November 13, release on November 20, and the demo on November 25. Confirm those targets today. The parallel integration in this version is Auth0.

MEETING FIELD GUIDE / 02

How RocketRide fits together

Keep the app, pipeline, node, and Redis server distinct. Each has a different job.

Term	Plain-language meaning
App	The screens a person opens: for example, a question box, answers, and cache statistics.
Pipeline	A workflow saved as a .pipe JSON file and executed by the RocketRide engine.
Node	A reusable capability in that workflow. Your proposed tool_redis node connects to Redis.
Agent	A model-driven loop that can decide to call tools. An app can also call certain tools directly.
Redis	A separate data service. The node sends it commands; the app and node do not implement Redis itself.
TTL	Time to live: how long a key should remain before Redis expires it.

What the Notion example teaches us

The inspected tool_notion node exposes operations as agent tools. Its manifest declares a tool with invoke capability and no data lanes. It is a useful structural reference for Redis, but its experimental status is not evidence that a new Redis node is production-ready. [3, 4]

File	What to look for
services.json	Node identity, tool capability, configuration fields, secure fields, and editor layout.
IGlobal.py	Read configuration, validate credentials, and manage shared lifecycle state.
IInstance.py	Expose typed operations using @tool_function and return structured results.
notion_client.py	Keep provider communication separate from the tool interface. Redis would use redis-py.
test_notion.py	Test behavior without a live engine by mocking external boundaries.

A useful discovery: RocketRide also has a Tools source. An application can call attached tools through client.tool() without an agent or LLM. This supports a deterministic cache lookup before answer generation. Confirm that this is the preferred architecture. [5]

The source snapshot already uses Redis for persistent memory and FalkorDB. That is a place to learn client usage; it does not replace the proposed general-purpose Redis tool. Its dependency lock pins redis-py to 7.4.1, matching the client brief. [3]

MEETING FIELD GUIDE / 03

A concrete first node scope

Proposed for sign-off. Operation names and limits below are design suggestions, not existing Redis-node APIs.

Purpose: expose a small, bounded set of Redis operations as RocketRide tools, reusable from an agent or from an app through a Tools-source pipeline. Support one standalone Redis deployment first; confirm hosted compatibility and server versions.

Proposed tool	Behavior and expected result
redis_get	Read a string value. A missing key is a successful result with found=false, not a connection error. JSON is encoded within strings.
redis_set	Save a UTF-8 string, with optional positive TTL. Return confirmation; the cache app always supplies expiration.
redis_delete	Delete explicitly named keys in a bounded batch and return the number removed. Requires writes enabled.
redis_ttl / redis_expire	Report remaining TTL and distinguish missing/no-expiry keys; optionally change expiration. Changing TTL is a write.
redis_increment	Increment an integer counter and return its new value. Define window-expiration behavior before implementing rate limits.
redis_scan	Find keys using a fixed configured prefix plus a narrower relative match. Return a continuation cursor and completion status.

Connection and configuration

Propose host, port, database index, optional ACL username/password, verified TLS, connection/read timeouts, a required key prefix, and a write-enable setting that defaults to false. Reuse RocketRide secret fields; credentials belong on the engine, never in browser code or committed pipeline files.

Boundaries from RocketRide's brief

Enforce the configured prefix for every key operation. Do not expose a generic raw-command tool. Do not expose FLUSHALL, FLUSHDB, KEYS, CONFIG, or EVAL. Publishing a change event can be stretch; subscribing requires a long-running listener and remains stretch. [1]

Proposed limits to discuss: reject empty prefixes and unrestricted key patterns; cap value size, requested batch size, scan calls, response size, and elapsed time. SCAN COUNT is a work hint, not a guaranteed result cap. Define continuation behavior without silently losing keys. [9]

Outside the proposed first version: binary payloads, rich Redis data types, cluster/Sentinel management, subscriptions, and general-purpose vector search. Confirm before excluding anything RocketRide actually requires.

MEETING FIELD GUIDE / 04

Three app ideas to bring

Lead with RocketRide's Answer Cache proposal, and use the other ideas as concrete alternatives.

App idea	What a user does	Why Redis is essential
1. Answer Cache **Recommended discussion lead**	Ask a question, see whether the response came from cache, inspect hit rate and estimated savings, and clear a topic as an authorized admin.	Caches answers with expiration, counts requests, and supports scoped invalidation. This is closest to the client brief.
2. Session Follow-up Assistant	Start a session, save selected facts, and ask a follow-up in a later workflow run. Inspect or clear that session.	Stores session state across runs with TTL. Requires an explicit session tool, since existing persistent-memory functionality overlaps.
3. Temporary Result Pickup	Run a workflow that creates a result, receive a lookup code, and retrieve the result later from another screen.	Shares an expiring value between workflow runs. Demonstrates read, write, expiration, missing keys, and deletion.

Suggested Answer Cache screens

• **Ask:** question input, answer, cache hit/miss label, and response time.

• **Session statistics:** hit rate, requests allowed, and estimated answer-generation cost avoided. Clearly label estimates.

• **Admin controls:** select an allowed topic, review the scope of deletion, and clear cached entries within that topic.

Separate app responsibilities from node responsibilities

The Redis node handles storage operations, prefixes, permission settings, and bounded I/O. The app or workflow handles question normalization, cache-key construction, topic selection, user identity, admin authorization, rate-limit policy, and cost reporting.

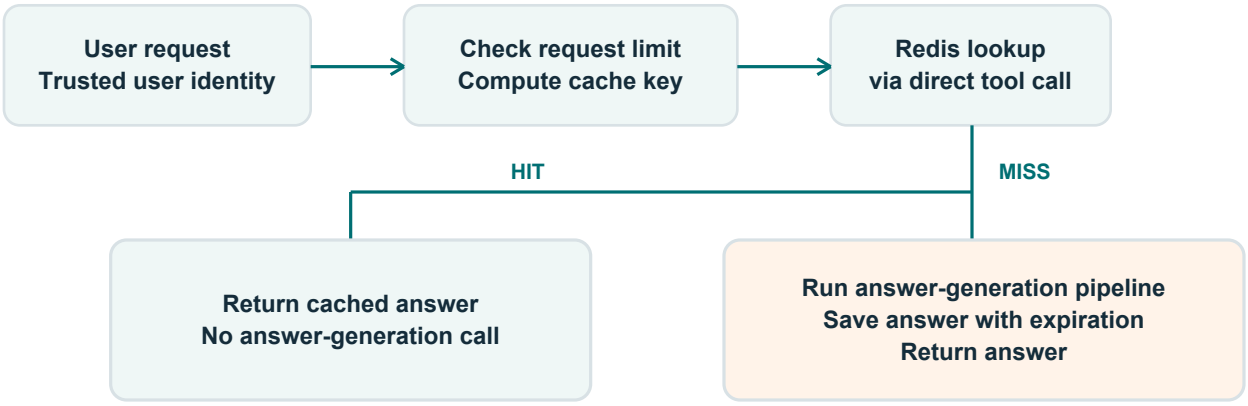
Recommendation: propose Answer Cache because it exercises the requested node meaningfully. Agree on its required features today. Bring the other two ideas to satisfy the request for alternatives; do not promise to build all three.

App Builder is RocketRide's extension workflow for scaffolding, previewing, packaging, and deploying shell apps. The inspected guide specifies pnpm and the official scaffold. Ask which template, engine, and developer namespace the team should use. [6]

MEETING FIELD GUIDE / 05

How Answer Cache should work

Use this page to explain the demo and catch the most important ambiguity before coding.



A repeated question should retrieve the stored answer. After expiration or an authorized topic clear, it should generate a fresh answer.

Resolve what "matching" means

The brief includes paraphrases under normalized matching, while embedding-based matching is stretch. Propose deterministic normalization first and confirm whether equivalent wording must also match.

Example to discuss: should "What is Redis?" and "Explain Redis to me" share an answer? What about "Does this expire?" and "Does this not expire?" Agree on a small accepted and rejected example set. Do not strip meaningful text to manufacture a hit.

Keep answers tied to the right context

Include user/sharing scope, topic, prompt/model version, normalized question, and relevant context in the key. A hash identifies content; it does not encrypt it. Keep personal answers isolated.

Be precise about the model-cost claim

A tool-only app lookup can avoid the answer-generation model call on a hit. If a model-driven agent decides whether to use Redis, that decision may itself require a model call. Ask whether RocketRide prefers app-controlled lookup, agent-controlled lookup, or both. [5]

Show actual hit/miss counts and observed model usage. State the baseline for estimated savings; a cache hit alone does not prove a cost reduction.

Rate-limit decision: define a fixed window or another algorithm, identify users from trusted platform identity, and test concurrent requests. Updating a counter and its expiry needs failure handling; separate INCR and EXPIRE calls can leave a counter without expiry. Agree on an implementation compatible with the no-EVAL boundary. [10]

MEETING FIELD GUIDE / 06

The testing plan

A draft your Integration & Test Lead can use in the design document after the team confirms the interface.

Test area	Evidence we should produce
Core operations	Write/read/delete round trips; missing and expired keys; TTL sentinel values; invalid types; counter increments.
Safety boundaries	Writes denied by default, including delete, increment, and TTL changes; every key stays under its prefix; no raw-command escape.
Bounded work	Value and batch limits; scan cursor continuation; empty scan pages and duplicates; deletion stops at its configured limits.
Connection failures	Bad credentials, timeout, unavailable Redis, verified TLS errors, and sanitized error messages without secrets.
Concurrency and retries	Overlapping increments and expiry; no automatic retry that could double an increment; clear behavior after uncertain writes.
RocketRide contract	Valid manifest and module paths; config fields; registered tool schemas; expected result and error shapes.
Live integration	Real Redis in Docker, then the node called through a local RocketRide engine. No fake backend substituted for this evidence.
App acceptance	Cache miss, hit, expiration, user isolation, limit/reset, authorized topic clear, and measurable generation-call behavior.

What was actually verified during preparation

78 Notion unit tests passed without a live engine, using the upstream test file. **2 Notion manifest contract tests passed**; 392 other contract cases were deselected. These are reference-node checks, not tests of a Redis node that has not yet been written.

```
python -m pytest nodes/test/tool_notion/test_notion.py \
--confcutdir=nodes/test/tool_notion -o addopts='' -q

python -m pytest nodes/test/test_contracts.py \
--noconftest -k tool_notion -o addopts='' -q
```

Commands were run from the inspected engine checkout with Python 3.12 and an isolated temporary environment containing pytest, pytest-asyncio, pytest-timeout, and requests. They intentionally bypass the full engine test harness. [3, 7]

Still to run: Docker Redis, a live RocketRide pipeline, the new node, and App Builder preview. Docker/redis-server were not available in the command environment. The extension is installed, but workspace trust is still required for its full runtime features.

MEETING FIELD GUIDE / 07

Responsibilities and milestones

Role names are proposed task owners. Personal assignments are deliberately blank for the team to fill in.

Role	Tasks owned	Name
Node Lead	Draft tool interface and design; implement node; own issue/PR and review fixes through merge.	_____
Integration & Test Lead	Reproduce environment; run local Redis; write test plan; own unit/contract/live tests and demo readiness.	_____
App Lead	Compare app ideas; sketch screens and pipeline; build through App Builder; run the live demo.	_____
Content Lead	Coordinate syncs and consent; own README/content coordination, blog and community-post drafts, and demo logistics.	_____

Dates in RocketRide's proposed plan

Date	Checkpoint	Dependency to confirm
Oct 30	Design approved	Scope decisions, app choice, owner names, named approver.
Nov 6	PR opened	Node, tests, README, reproducible local verification.
Nov 13	PR merged	Reviewer availability and review fixes; blog draft review.
Nov 20	Node released; app built	RocketRide release process; app and content review.
Nov 23-24	Blog and forum posts	Released node link; content approval and publishing owners.
Nov 25	Live demo and recording	Released node verified; App Builder preview fallback ready.
Early December	Clips and social posts	Demo recording and consent; RocketRide edits clips.

The brief sets a weekly 15-minute engineering sync, a named PR reviewer, Discord support, and a separate AIC Board check-in. It describes November 26-30 as late-fix buffer, with no new scope. Confirm ownership of release notes and the docs-site entry. [1]

Release dependency: the brief requires the catalog release before hosted app publication. If release slips, use App Builder preview for the demo and wait to publish release-linked posts. Confirm the demo environment, Redis hosting, and any model budget. [1, 8]

MEETING FIELD GUIDE / 08

Use this during the meeting

Suggested agenda for a 30-minute call: align on scope, settle the app, confirm delivery, then read back decisions.

Time	Discussion
0-5 min	State your understanding; confirm which parts of the PDF are fixed and which are proposals.
5-13 min	Review node operations, access controls, limits, and the reference implementation.
13-21 min	Choose the app; resolve matching and model-call behavior; confirm success criteria.
21-27 min	Confirm developer setup, test harness, reviewer, owners, and milestone dates.
27-30 min	Read back decisions and next actions. Record who approves the design.

The questions that change implementation

1. Which PDF requirements are fixed, and which can we simplify before design approval on October 30?

2. Can we agree on the exact first-version operations, connection options, and supported Redis deployment?

3. Is Answer Cache the selected app? Which of normalization, rate limiting, topic clear, and statistics are required?

4. What paraphrase examples must match? Is semantic equivalence required before the stretch phase?

5. May the app use the Tools source for deterministic cache lookup, with generation only on a miss? Must an agent also demonstrate the node?

6. Which branch/release, example node, contract tests, and local-node loading process should we follow?

7. What is the trusted user/admin identity source, and who supplies development secrets, model access, and demo Redis?

8. Who approves design, reviews the PR, releases the node, and publishes the documentation? Are the dates realistic?

A short opening you can read aloud

> "We reviewed your brief and the Notion node. Our first proposal is a bounded Redis tool for values, expiration, counters, and scoped key discovery and deletion. We brought Answer Cache and two alternative app concepts. We want to confirm the required operations, matching behavior, app architecture, and delivery dates before finalizing the design."

Read back before leaving: selected app _____; required operations _____; matching rule _____; owners _____; design approver _____; reviewer _____; next deadline _____.

MEETING FIELD GUIDE / 09

Your VS Code walkthrough

The original brief, actual upstream source, and this new PDF are opened together for inspection.

Keep the two repositories distinct. Your existing rocket-ride folder contains the earlier GitHub issue-to-SQLite exercise. The actual RocketRide engine is the separate rocketride-server checkout. This packet adds no Redis implementation to either repository.

Open in this order	What to learn
1. This PDF, then the original client PDF	Understand the proposal, then compare it with RocketRide's wording and dates.
2. Engine README.md	Understand the product and the extension/engine relationship.
3. tool_notion/services.json	Find classType, invoke capability, empty lanes, secure fields, and the configuration shape.
4. tool_notion/IGlobal.py	See where credentials are read and validated. Redis connection setup would follow the node lifecycle.
5. tool_notion/IInstance.py	Find @tool_function, its input/output schema, argument normalization, and structured errors.
6. tool_notion/notion_client.py	See why provider-specific I/O can be separated from tool registration.
7. test_notion.py and test_contracts.py	Compare mocked behavior tests with manifest/interface checks. Neither proves a live Redis integration.
8. Tools source README	See direct client.tool() calls without a model. This is the proposed cache lookup route.
9. Node testing and App Builder guides	See official test commands and the supported app creation/preview/publish loop.

What is ready on this computer

The official RocketRide VS Code extension v1.5.0 and a PDF viewer were installed. The source checkout is a clean sparse reference copy on develop at commit 40ea9ecbbfabbed7d4d7250415ef8b769be5a0bc. It includes nodes, relevant docs, extension source, scripts, examples, and selected shared code.

The workspace is open in Restricted Mode for source browsing. Live extension/runtime features have not been enabled. A full engine build, Docker Redis, and App Builder preview have not been run. You can read all opened source and the separate PDFs without mistaking exploration for a completed implementation.

MEETING FIELD GUIDE / 10

Evidence and source map

Client requirements come from the supplied brief and Discord screenshot. Proposed implementation choices remain open for sign-off.

[1] RocketRide client brief
Supplied RocketRide_AIC_Redis_Node_Integration_v2.pdf, 7 pages. Scope and safety: p. 1. App/roles: pp. 1-3. Dates, release dependencies, outreach: pp. 4-7.

[2] Discord preparation request
Supplied screenshot, message dated October 27, 2026. Requests project review, product/repository exploration, node scope, 2-3 app ideas, and task owners. Says the proposal can be refined together.

[3] Inspected engine repository
develop snapshot 40ea9ecbbfabbed7d4d7250415ef8b769be5a0bc. Code structure, existing Redis users, dependency lock, contributor process, and reference tests.

[4] Notion node documentation
Reference tool shape and its stated experimental status.

[5] Tools source documentation
Direct tool invocation from a client, without an agent or LLM.

[6] App Builder guide in inspected source
Scaffolding, pnpm, preview, package, deploy, and publish behavior.

[7] Node testing guide in inspected source
Contract versus live integration tests and official builder commands.

[8] Upstream contribution requirements
PRs target develop, link an issue, follow repository branch naming, and include tests/docs. Catalog restrictions affect hosted apps.

[9] Redis SCAN reference
Cursor iteration, duplicate/empty results, and COUNT as a hint rather than a hard result limit.

[10] Redis INCR reference
Counter behavior and the documented INCR/EXPIRE rate-limiter race.

Questions to keep explicit

Final tool names, payload/work limits, supported Redis versions, error contracts, rate-limit semantics, topic-clear consistency, matching rules, user/admin authorization, app namespace, reviewer, and release dates all need agreement. Unnamed role fields remain blank at your request.

Prepared October 29, 2026. The original client PDF and upstream source files were not edited. No messages were sent, issues opened, or scope commitments made on the team's behalf.